BCR-6605 – everything still to do

Seven steps, in order. Each one is exact – file, place, FIND and REPLACE. Parts F and G are not in this document; they come later.

#	File	What
1	HealthAlarmsPreferenceStore	**NPE** – a `continue;` swallowed into a comment
2	HealthAlarmsPreferenceStore	Popup never fires – the name list is never filled
3	HealthAlarmsComposite	Remove the duplicate dead field
4	HealthCommonBusiness	**NPE** – loop outside its null guard
5	the filtering construction site	One line: hand the deviations to the algorithm
6	AbstractFilteringRuleAlgorithm	Filtering law from the deviation (REQ1)
7	persistence.xml	`ddl-generation` → `none`

Already done – do not repeat: the column split in `WCTreeCellModifier`, the Points column end to end (enum, cell modifier, label provider, cell editor), `innerSave()` baseline handling, `hasDeviationFieldChanged`, `toDeviationDto`, the `maxPointsPerHour` field on entity and DTO, the `HealthCommonBusiness` mapping, both database columns, the graph overlay, and the Points override in `AbstractFilteringRuleAlgorithm`.

1 – the swallowed continue (NPE)

File: HealthAlarmsPreferenceStore **Method:** `innerSave()`

The `continue;` ended up inside the comment, so the guard does nothing and `thresholdModels.get(indicator)` returns null on the next line. Any inactive indicator throws `NullPointerException` on Apply.

FIND

```
            if (!thresholdModels.containsKey(indicator)) {
               // This indicator is likely inactive and has no current configuration to copy continue;
            }
```

REPLACE

```
            if (!thresholdModels.containsKey(indicator)) {
               // This indicator is likely inactive and has no current configuration to copy
               continue;
            }
```

2 – the "Deviation created" popup never appears

File: HealthAlarmsPreferenceStore **Method:** `saveDeviations()`

`lastSavedDeviationIndicatorNames` is declared and returned but never written to, so `performApply()` always sees an empty list. Names are collected locally and copied in only after the server call succeeds, so the popup cannot announce a deviation that failed to save.

REPLACE the whole method with

```
   public void saveDeviations() {
      List<IndicatorDeviationDto> newDeviations = new ArrayList<>();
      List<String> savedNames = new ArrayList<>();
      Aircraft deviationAircraft = null;
      Date now = new Date();
      lastSavedDeviationIndicatorNames.clear();
      for (Entry<Aircraft, WCMechanicalComponentModel> entry : data.entrySet()) {
         deviationAircraft = entry.getKey();
         Multimap<Indicator, WCThresholdModel> thresholds = getComponentThresholds(entry.getValue());

         for (WCThresholdModel model : thresholds.values()) {
            LOGGER.info(" Value {} deviated={} deviationId={} icId={} te={}", model.getIndicator().getName(), model.isDeviated(),
model.getDeviationId(), model.getIndicatorConfigurationId(),
                  model.getTeReference());

            if (!model.isDeviated() || model.getDeviationId() != null) {
               continue;
            }
            if (!hasDeviationFieldChanged(model)) {
               continue;
            }

            newDeviations.add(toDeviationDto(model, now));

            // BCR-6605 remember the indicator for the "Deviation created" popup
            String indicatorName = getIndicatorDisplayName(model.getIndicator().getId());
            if (indicatorName.isEmpty()) {
               indicatorName = model.getIndicator().getName();
            }
            if (!savedNames.contains(indicatorName)) {
               savedNames.add(indicatorName);
            }
         }
      }
      if (!newDeviations.isEmpty() && deviationAircraft != null) {
         try {
            healthBusiness.createDeviations(deviationAircraft.getId(), MgsSession.getInstance().getUser().getLoginName(), newDeviations);
            // Only announce what the server actually accepted
            lastSavedDeviationIndicatorNames.addAll(savedNames);
         } catch (Exception e) {
            LOGGER.error("Could not save deviations", e);
```

```
            }
        }
    }
```

3 – remove the duplicate dead field

File: HealthAlarmsComposite **Where:** the last lines before the closing brace

Added to the composite by mistake. The real ones live in the store, and `performApply()` already calls the store's version. Nothing reads these.

DELETE

```
   private final List<String> lastSavedDeviationIndicatorNames = new ArrayList<>();

   public List<String> getLastSavedDeviationIndicatorNames() {
      return lastSavedDeviationIndicatorNames;
   }
```

Then Ctrl+Shift+O – `java.util.ArrayList` may become unused.

4 – filtering law NPE

File: HealthCommonBusiness **Method:** `toDtos(...)`

The loop reads `d.getFilteringLaw().getContextualParameters()` *outside* the null check directly above it. The first deviation without a filtering law throws, and that breaks both deviation tabs.

FIND

```
         if (d.getFilteringLaw() != null) {
            dto.setFilteringLawId(d.getFilteringLaw().getId());
            dto.setFilteringLawName(d.getFilteringLaw().getName());
         }
         for (ContextualParameter p : d.getFilteringLaw().getContextualParameters()) {
            if (params.length() > 0) {
               params.append("  |  ");
            }
            params.append(p.getName()).append(", min=").append(p.getMinValue()).append(", max=").append(p.getMaxValue());
            if (p.getThresholdOfVariabilityValue() != null) {
               params.append(", S=").append(p.getThresholdOfVariabilityValue());
            }
         }
```

REPLACE

```
         if (d.getFilteringLaw() != null) {
            dto.setFilteringLawId(d.getFilteringLaw().getId());
            dto.setFilteringLawName(d.getFilteringLaw().getName());
            // BCR-6605 this loop must stay inside the null check
            for (ContextualParameter p : d.getFilteringLaw().getContextualParameters()) {
               if (params.length() > 0) {
                  params.append("  |  ");
               }
               params.append(p.getName()).append(", min=").append(p.getMinValue()).append(", max=").append(p.getMaxValue());
               if (p.getThresholdOfVariabilityValue() != null) {
                  params.append(", S=").append(p.getThresholdOfVariabilityValue());
               }
            }
         }
```

5 – hand the deviations to the filtering algorithm

How to find it: Ctrl+H → File Search → `new HelionixFilteringPerFileAlgorithm`, then `new MarmsFilteringPerFlightHoursAlgorithm`. One or two hits, same class.

The algorithm now holds the deviations, but nothing gives them to it. Immediately after the object is created, add:

```
      // BCR-6605 let a deviated Points value and filtering law override the configuration
      filteringAlgorithm.setDeviations(healthHistoryProvider.getDeviations());
```

Use the real local variable name instead of `filteringAlgorithm`. For the provider:

If this is in scope there	Use
`IHealthHistoryProvider healthHistoryProvider`	`healthHistoryProvider.getDeviations()`
Not in scope, but the class calls `LearnedThresholdAnalyzer.updateLearnedThresholds(...)`	The provider is an argument of that call – reuse the same reference
Neither	Skip the line. Points and the filtering law are still stored, just not applied during filtering. Nothing breaks

The map is already filtered by the flight's session date inside `getDeviations()`, so there is no date rule to repeat here.

6 – filtering law from the deviation

File: AbstractFilteringRuleAlgorithm **Method:** `getFilteredValues(...)`

REQ1 lists the filtering law as a deviation field. It is stored and displayed but the analyser still reads the configured one. Same pattern as Points, in the same class.

FIND

```
            if (indicatorConfiguration.getFilteringLaw() != null && isFilteringLawApplicable(indicatorValueAdapters)) {
               applyFilteringLawAlgorithm(indicatorValueAdapters, filteringLaws, indicatorConfiguration.getFilteringLaw());
            }
```

REPLACE

```
            // BCR-6605 a deviation can replace the configured filtering law
            final String filteringLawName = getFilteringLawName(indicatorConfiguration);
            if (filteringLawName != null && isFilteringLawApplicable(indicatorValueAdapters)) {
               applyFilteringLawAlgorithm(indicatorValueAdapters, filteringLaws, filteringLawName);
            }
```

Add this method beside `getDeviatedPoints`

```
   /**
    * BCR-6605 the filtering law to apply: the deviated one when a deviation set it, otherwise the
    * configured one. Like Points, the law belongs to the indicator, so the first severity carrying a
    * deviated law wins.
    *
    * @param conf
    *           the indicator configuration being filtered
    * @return the law name to apply, or null when there is none
    */
   private String getFilteringLawName(final IndicatorConfiguration conf) {
      if (!deviations.isEmpty() && conf.getIndicator() != null) {
         for (Severity severity : Severity.values()) {
            IndicatorDeviation deviation = deviations.get(conf.getIndicator().getId() + "|" + severity.name());
            if (deviation != null && deviation.getFilteringLaw() != null) {
               return deviation.getFilteringLaw().getName();
            }
         }
      }
      return conf.getFilteringLaw();
   }
```

No new imports. With no deviation the method returns `conf.getFilteringLaw()`, so behaviour is unchanged.

7 – stop the redeploy wiping the database

File: the server `persistence.xml`

This is why the test data vanished on 13-10. Every redeploy drops and recreates every table.

FIND

```
<property name="eclipselink.ddl-generation" value="drop-and-create-tables"/>
```

REPLACE

```
<property name="eclipselink.ddl-generation" value="none"/>
```

Both deviation columns already exist, so nothing needs auto-creating. Do this last.

Test

#	Action	Expect
1	Rebuild client and server, redeploy	–
2	Edit **Level** only, add TE, Apply	"Deviation created on indicator xxx". No flight popup, no configuration row
3	Edit **Points**, add TE, Apply	Same. Both severity rows move together
4	Edit **Filtering Law**, add TE, Apply	Same
5	`SELECT te_reference, max_points_per_hour, filtering_law_id FROM public.indicator_deviation ORDER BY start_date DESC LIMIT 3;`	All three stored
6	Edit **Value** only, Apply	Flight popup only, no deviation row
7	Edit **Max time without acquisition** only, Apply	Flight popup, configuration row
8	Edit **Level** and **Value** together, Apply	Both popups. The new configuration carries the **old** Level
9	Open the Deviation list tab	No NPE, rows listed even without a filtering law
10	Deviated row	Green and not editable
11	Download a flight on a deviated indicator	Acquisitions kept per hour follow the **deviated** Points; the deviated law is applied
12	Redeploy, then re-run step 5	Rows still there – proves step 7